



An Edge-Native Architecture for a Unified, Multi-Tenant Queue Management System with Adaptive PWAs and Hybrid User Support

This report provides a comprehensive deep-dive analysis into the design and technical implementation of a sophisticated queue management Software-as-a-Service (SaaS) platform. The proposed architecture centers on a single Progressive Web App (PWA) built upon Cloudflare's edge computing ecosystem, designed to serve a diverse user base across multiple organizational structures. It addresses the core requirements of supporting both standard and hierarchical tenant models, ensuring seamless experiences for smartphone and non-smartphone users, and implementing robust security and real-time synchronization mechanisms. The analysis synthesizes insights from architectural blueprints, official documentation, and expert discussions to present a detailed plan for building a scalable, secure, and highly performant system.

Core Architecture: A Unified Edge-Native PWA

The foundational decision to construct the entire platform around a single, unified Progressive Web App (PWA) represents a strategic choice that maximizes development efficiency, simplifies maintenance, and ensures global consistency [51](#). Instead of managing separate codebases for web, iOS, and Android applications, a single PWA code bundle can be deployed globally, delivering a consistent user experience regardless of device or operating system [30](#). This approach significantly reduces development costs and accelerates feature deployment cycles [41](#). The entire architecture is engineered to leverage Cloudflare's global edge network, which serves as the central nervous system for delivering low-latency, high-concurrency performance essential for real-time queue management [20](#).

The core of this architecture is a serverless compute layer powered by **Cloudflare Workers**. These microsecond cold-start functions execute at the edge—across hundreds

of data centers worldwide—processing API requests, handling authentication logic, and acting as a gateway to other stateful services [20](#) [77](#) . By running application logic closer to the end-user, the platform inherently minimizes network latency and avoids the bottlenecks common in traditional centralized architectures, making it exceptionally resilient to traffic spikes [49](#) . Frontend assets—including HTML, JavaScript, CSS, images, and the Service Worker script—are deployed via **Cloudflare Pages** or cached in **Workers KV**, a globally distributed key-value store [30](#) . This ensures that the PWA loads in under 100 milliseconds anywhere on the planet, providing an instant-on experience even in areas with poor network connectivity [30](#) . The ability to use a single JavaScript bundle for both client-side execution in the browser and server-side execution within Cloudflare Workers further streamlines the development workflow, enabling true isomorphic behavior where the same code handles online requests on the edge and offline requests via the Service Worker cache [51](#) .

For stateful operations, particularly atomic counter increments and real-time broadcasting, the architecture relies on **Cloudflare Durable Objects**. These provide a single-threaded, strongly consistent execution environment for specific tasks, effectively creating isolated, persistent objects co-located with users [20](#) [77](#) . Each serving location, whether a Standard Tenant or a Sub-Tenant, is assigned its own unique Durable Object instance. This design guarantees perfect serialization of ticket numbers and serves as a low-latency hub for broadcasting WebSocket messages to all connected clients for that specific location [2](#) [19](#) . For data persistence, **Cloudflare D1** serves as the serverless SQLite database. D1 is designed for horizontal scaling, allowing the platform to efficiently manage a large number of tenants, with each potentially having its own dedicated 10GB database instance [23](#) [24](#) . This makes a physically isolated, database-per-tenant strategy economically viable and highly secure, especially for enterprise clients in regulated industries [21](#) . Finally, on the client side, a Service Worker combined with IndexedDB implements the offline-first principle. The Service Worker caches frontend assets, ensuring the app shell loads instantly from local storage, while IndexedDB stores critical session data like the user's ticket information, facilitating a seamless experience when users refresh their browser, close the tab, or temporarily lose network connection [51](#) .

Data Isolation and Multi-Tenant Governance

To support both a standard three-tier structure (Platform Admin, Business Tenant, End User) and an extended four-tier hierarchical model (Platform Admin, Master Tenant, Sub-Tenant, End User), the platform requires a flexible yet robust multi-tenancy governance strategy [21](#). The foundation of this strategy lies in a hybrid data isolation model combining logical row-level security with physical database separation, all managed through a carefully designed relational schema in Cloudflare D1. This approach ensures strict data boundaries, prevents cross-tenant leakage, and accommodates diverse business needs, from single-location businesses to large corporate franchises [26](#) [28](#).

The database schema is designed for maximum flexibility. A central `tenants` table stores metadata for every business entity. Critically, this table includes an optional `master_tenant_id` column that acts as a foreign key referencing the `id` of another record in the same table [21](#). For a Standard Tenant (a single business), this field is `NULL`. For a Sub-Tenant (a branch or franchise location), this field contains the `id` of its parent Master Tenant, thereby establishing the hierarchical relationship cleanly within the database structure [21](#). All entities that require tenant-specific data, such as queues, tickets, and staff members, will have a `tenant_id` column that references the `id` in the `tenants` table. This ensures that every piece of operational data is unambiguously tied to its specific serving location, which could be either a top-level Standard Tenant or a lower-level Sub-Tenant [21](#).

Enforcement of these data boundaries occurs at two levels: logical and physical. Logical isolation is achieved by ensuring that every query executed against shared tables always filters results using the `tenant_id` of the currently authenticated user's context [26](#). While efficient, this method carries the risk of data leaks if developers forget to include the necessary filter, a bug that can have severe consequences [28](#). To mitigate this, the architecture mandates that all data access from the frontend dashboard PWA must pass through edge middleware in Cloudflare Workers, which enforces tenant scope before any handler logic is executed [36](#). Physical isolation provides a stronger guarantee. For tenants requiring higher security or regulatory compliance (e.g., healthcare, finance), the platform can be configured to provision a dedicated D1 database per tenant [21](#). Since each D1 database is a discrete SQLite instance, queries originating from one tenant's context are physically unable to access data belonging to another, providing a robust architectural barrier [21](#). The economics of this approach on Cloudflare are highly favorable; a thousand D1 databases with modest usage can cost roughly \$10 per month, making this level of isolation far more accessible than on traditional cloud providers [21](#).

State isolation is handled naturally by the nature of Cloudflare Durable Objects. Each DO instance operates in its own single-threaded environment and cannot share memory or accidentally access the data of another object [56](#) [77](#). By creating a unique DO ID for each serving location (using a naming convention like `tenant:{tenant_id}`), the platform ensures that the state (e.g., the current queue position) for one location is completely isolated from others, preventing race conditions and accidental state corruption [21](#). This combination of a flexible schema, logical filtering enforced at the edge, and physical database separation creates a secure, scalable, and future-proof multi-tenancy architecture that can seamlessly accommodate both simple and complex business structures.

Feature	Standard Three-Tier Model	Extended Four-Tier Model
Organizational Structure	Platform Admin → Business Tenant → End User	Platform Admin → Master Tenant → Sub-Tenant → End User
Tenant Relationship	All tenants are peers; no hierarchy.	Sub-Tenants are children of a Master Tenant.
Database Schema	The <code>tenants</code> table uses <code>NULL</code> for <code>master_tenant_id</code> 21 .	The <code>tenants</code> table links <code>id</code> to <code>master_tenant_id</code> 21 .
Use Case	Single restaurants, clinics, banks, or small businesses 41 .	Franchises, enterprise clients, or government bodies managing multiple branches 21 .
Management Scope	Branch Manager controls only their single location.	Master Tenant can manage all Sub-Tenants and view corporate-wide analytics.

Real-Time State Management and Synchronization

The core functionality of a queue management system hinges on its ability to handle massive concurrency and synchronize state in real time without errors. The proposed architecture, centered on Cloudflare Durable Objects, is exceptionally well-suited to address two of the most critical challenges in this domain: generating a perfectly sequential, non-duplicate ticket number for thousands of simultaneous users, and broadcasting live updates to all connected clients instantaneously [19](#) [50](#). This is achieved through a combination of atomic operations and bi-directional WebSocket communication, all managed at the network's edge.

The problem of race conditions arises when numerous customers scan the same QR code at nearly the exact same moment. A naive implementation that reads the last ticket number, increments it in the application code, and writes it back would inevitably lead to duplicates. The solution is to perform the increment operation atomically within the

database itself. While Redis with its `INCR` command is a common solution for this type of counter, the integrated Cloudflare ecosystem offers a superior alternative: Durable Objects [2](#) [19](#). The recommended approach is to assign a unique Durable Object to each individual serving location (identified by its `tenant_id`). When a new ticket request arrives at a Cloudflare Worker, the Worker routes the request to the corresponding tenant's Durable Object. Inside this single-threaded, strongly consistent object, the counter is incremented atomically, ensuring that requests are processed sequentially and without interference [56](#) [77](#). The next unique serial number is then returned to the worker, which saves the new ticket record in the D1 database and sends the confirmation to the end-user's PWA [19](#). This method eliminates external dependencies, simplifies the architecture, and leverages Cloudflare's native primitives for stateful coordination [42](#). An alternative using D1's native SQLite transactions (`SELECT . . . FOR UPDATE`) is also viable but may be less performant under extreme write loads compared to the purpose-built Durable Object [21](#).

For real-time broadcasting, the platform utilizes WebSockets, which provide a persistent, low-latency, bi-directional communication channel between the client and server [58](#). Cloudflare Durable Objects natively support accepting WebSocket connections, transforming them into powerful, low-latency broadcast hubs [14](#) [48](#). When a staff member calls the "next" customer from their dashboard, the action triggers a function that executes within the relevant tenant's Durable Object. This object then broadcasts a small JSON payload (e.g., `{"current_called": 42}`) to all active WebSocket connections associated with that specific tenant ID [75](#). This includes the end-user's mobile device, the agent's dashboard, and the wall-mounted TV display. Because the Durable Object and its WebSocket connections are hosted at the edge, this broadcast happens with minimal latency, ensuring that everyone—the customer watching their phone, the agent at the counter, and the people waiting in the lobby—sees the update simultaneously [50](#). This is vastly more efficient than a polling-based approach, which would waste bandwidth and introduce noticeable delays. The Hibernation API can be used to put idle Durable Objects to sleep, reducing costs while maintaining the ability to wake them up instantly when a new event occurs [15](#) [47](#). This combination of atomic counters and WebSocket broadcasting forms the real-time engine of the platform, delivering the responsive, synchronized experience that is critical for effective queue management.

End-User Experience: Inclusivity and Frictionless Onboarding

A successful queue management platform must prioritize a frictionless and inclusive end-user experience, accommodating a wide spectrum of technological proficiency and device availability. The chosen PWA architecture, combined with thoughtful onboarding flows and alternative interaction methods, enables the creation of a single, versatile application that feels native and intuitive for every customer, from a tech-savvy smartphone owner to an elderly individual with a basic feature phone.

The primary onboarding flow begins when a customer scans a QR code at a business location. This QR code directs the user's browser to a public-facing route, such as `/join`, within the PWA [63](#). From there, the platform employs OS-adaptive strategies to guide the user toward installing the PWA for a better experience, while gracefully degrading for those who decline. On **Android devices**, the PWA's JavaScript code listens for the `beforeinstallprompt` event. This event is fired by Chrome when the browser determines the site meets installability criteria [11](#). The event is captured and stored in a variable, but its default behavior is prevented [10](#). The user is presented with a clear call-to-action button, for example, "Join Queue & Install App." When the user clicks this button, the saved `deferredPrompt` is programmatically triggered using the `prompt()` method, displaying the native browser installation dialog immediately and tying the install action to a positive user intent [13](#) [64](#). This creates a seamless, almost instantaneous installation experience.

On **iOS devices**, the situation is different due to Safari's limitations. Apple does not support the automated `beforeinstallprompt` event, so the installation process must be guided manually [57](#) [72](#). Upon loading the `/join` page, the PWA's JavaScript detects the iOS environment. If detected, it displays a prominent, animated tooltip or banner at the bottom of the screen, complete with a visual arrow pointing to the Share icon in the browser's address bar. The message instructs the user: "To track your spot live, tap [Share Icon] and select 'Add to Home Screen'" [70](#) [71](#). This guided instruction makes the manual process foolproof for users unfamiliar with the procedure. Crucially, this entire installation process is non-blocking. If a user declines the prompt on Android or ignores the instructions on iOS, the platform's "No-Install Required" safeguard ensures they can still join the queue and track their status directly within the browser tab [30](#). To maintain session continuity, the backend issues a unique ticket token upon check-in, which is securely stored in the browser's `localStorage` [30](#). This allows the user to close the tab, lock their phone, or even lose cellular signal temporarily, and upon reopening the URL,

the PWA will instantly restore their ticket details, preventing them from needing to re-scan the QR code [30](#).

Beyond smartphone users, the platform is designed for inclusivity by offering multiple alternative check-in methods. For customers without personal devices, businesses can deploy a **staff-assisted manual check-in**. Counter staff can add walk-in customers directly through their agent interface, who are then seamlessly integrated into the queue with the next available consecutive number, maintaining perfect synchronization with users who scanned QR codes [4](#). Another powerful option is the **self-service kiosk**. A tablet locked into a dedicated `/kiosk` route of the PWA can function as a digital concierge, allowing customers to enter their name and join the queue themselves [6](#). This kiosk can be paired with a thermal printer to issue a physical paper ticket, providing a tangible reference for customers who prefer it [8](#). Furthermore, for customers with older feature phones that lack internet access, the system supports **SMS-based status updates**. During check-in, either by kiosk or staff, the customer's phone number can be collected. The Cloudflare Worker can be configured to receive incoming SMS messages via a Twilio webhook, allowing the system to send text message updates about their queue position, bridging the gap between modern web technology and legacy mobile networks [52](#).

Hardware Integration and Alternative Check-In Methods

Providing robust alternatives for customers without smartphones is a cornerstone of an inclusive queue management system. Beyond the software-based solutions of staff-assisted check-ins and SMS updates, the platform can integrate with physical hardware to create a seamless, self-service experience for walk-up customers. The most prominent of these is the integration of thermal receipt printers into a self-service kiosk setup, a feature that transforms a standard tablet into a fully functional digital concierge station [9](#).

Implementing a thermal printer requires careful consideration of connectivity and reliability. Modern kiosk printers offer various connectivity options, including USB, Serial, and Ethernet LAN, which allows for flexible integration into different business environments [9](#). However, direct browser-to-printer communication presents challenges. While some modern browsers offer APIs for interacting with USB devices, these are often limited and inconsistent across platforms. A more reliable approach

involves using a local print server or leveraging a cloud-based print service. One solution is to build a lightweight local print server using a device like a Raspberry Pi, which connects to the thermal printer via USB and exposes a simple HTTP endpoint for receiving print jobs 39 . The PWA on the kiosk would then send the ticket data to this local server's IP address instead of trying to communicate with the printer directly. This isolates the hardware dependency and keeps the kiosk UI clean and simple.

An alternative to a self-hosted server is to partner with a cloud print service provider. Services like Expedy Print or Star Micronics' offerings provide a cloud print API that allows any backend to send print jobs to remote thermal printers over the internet 7 33 . The kiosk's PWA would make a standard HTTP POST request to the cloud print service's API, passing the ticket data in the payload. The service then handles the secure routing and formatting of the job to the designated printer at the customer's location 60 . This approach abstracts away the complexities of networking, driver installation, and hardware maintenance, shifting that responsibility to the service provider. It simplifies the setup for the end business but may involve subscription fees and introduces a third-party dependency. Regardless of the method chosen, the system design must account for potential failures, such as a disconnected printer or network outage. The kiosk interface should provide clear feedback to the user, perhaps by showing a virtual ticket on the screen even if the physical print fails, ensuring the customer still receives confirmation of their place in line.

The following table outlines the key characteristics and trade-offs of different hardware integration approaches:

Integration Method	Description	Pros	Cons
Local Print Server (e.g., Raspberry Pi)	A small computer (like a Raspberry Pi) sits between the tablet and the printer, converting web requests to commands for the USB-connected printer 39 .	High reliability, full control over the hardware, no recurring subscription fees.	Requires initial setup and maintenance effort; adds a hardware component to the kiosk.
Cloud Print Service (e.g., Expedy Print)	The PWA sends a print job to a cloud-based API, which then forwards the job to the physical printer over the internet 33 60 .	Simple setup, vendor manages hardware/cloud infrastructure, cross-platform compatibility.	Potential for recurring subscription costs, dependent on third-party service availability and reliability.
Direct Browser Printing	The PWA attempts to use browser APIs (e.g., WebUSB) to communicate directly with the printer connected to the tablet 32 .	No additional hardware or servers needed; theoretically simplest.	Highly unreliable, inconsistent support across browsers and operating systems, difficult to debug.

Ultimately, offering a pre-tested integration with popular kiosk printer brands from manufacturers like Star Micronics would provide businesses with a plug-and-play

solution, significantly lowering the barrier to adoption [7](#) . By providing clear documentation and multiple configuration paths, the platform can empower businesses to choose the hardware integration strategy that best fits their budget, technical expertise, and reliability requirements.

Security Enforcement and Risk Mitigation

While the proposed architecture is powerful, its success depends on rigorous security enforcement and proactive mitigation of inherent platform risks. The multi-tenant nature of the SaaS requires a defense-in-depth strategy, with security measures implemented at every layer of the stack—from the client-side UI down to the edge compute and database layers. Key risks include the nuances of iOS PWA storage, the complexity of hardware integration, and the long-term operational aspects of managing stateful primitives at scale.

The most critical security principle is to never trust the client. Frontend logic can be manipulated, so UI elements and navigation paths must be conditionally rendered based on a user's role, which is determined by a JSON Web Token (JWT) stored in `localStorage` [41](#) . However, this conditional rendering is solely for user experience and is not a security mechanism. Absolute security is enforced at the edge using Cloudflare Worker middleware. Every authenticated request from the dashboard PWA to a protected API endpoint must include the JWT in its `Authorization` header [53](#) . The Worker middleware intercepts this request, cryptographically verifies the token's signature using libraries like `jose` or Cloudflare's API Shield, and decodes its payload [53](#) [54](#) . The payload contains the user's role and `tenant_id`. The middleware then performs a strict authorization check: does this user's role have permission to access the requested resource within their specific tenant boundary? If a user with a "Branch Manager" role attempts to access an API path reserved for a "Master Tenant," the Worker immediately rejects the request with a `403 Forbidden` error, preventing any attempt to bypass security checks [36](#) .

One significant and subtle risk stems from the behavior of PWAs on iOS. When a user adds a PWA to their home screen in Safari, it runs in a separate storage sandbox from the main browser [67](#) . This means that data written to `localStorage` while the PWA is open from the home screen icon is *not* accessible if the user later opens the same URL from within a standard Safari tab [67](#) [68](#) . This breaks the "session continuity" guarantee. A

user who installs the PWA and then loses their network connection might find their ticket has disappeared if they try to re-open the link from their browser history or bookmarks. The primary mitigation for this is user education and workflow design. The onboarding tooltip on iOS should explicitly state that for the best experience, the user should always open the app from the icon on their home screen. While technically challenging, this limitation underscores the importance of designing the user journey to favor the installed PWA over the standalone browser version.

Another area requiring careful management is hardware integration, specifically for thermal printers. This introduces dependencies on specific hardware models, drivers, and network configurations, which can become a major point of failure and increase customer support overhead [32](#). The risk of printer jams, network disconnections, or driver conflicts can negatively impact the customer experience. The mitigation strategy is twofold: first, provide pre-tested integrations with a curated list of reliable kiosk printers from established vendors like Star Micronics [7](#). Second, offer businesses a choice between a simpler, cloud-based print service that abstracts away hardware complexities and a documented guide for setting up a self-hosted local print server for greater control [33](#) [39](#). Clear communication of these options during onboarding is crucial for setting expectations.

Finally, while Durable Objects are designed to scale horizontally, each individual object is single-threaded [56](#). As the platform scales to tens of thousands of locations, the operational management of a vast number of active DO instances could become a concern. Although Cloudflare Platforms offers features like hibernation to reduce costs on idle objects, the total number of active DOs is subject to limits [15](#). The long-term mitigation for this is to monitor usage patterns closely and aggressively employ the WebSocket Hibernation API to put inactive objects to sleep [47](#). The architecture should be designed to allow for future migration to a more scalable coordination mechanism if necessary, though this is likely a long-term consideration rather than an immediate concern. By anticipating these risks and implementing layered mitigations, the platform can deliver on its promise of being secure, reliable, and scalable from day one.

Reference

1. State Synchronization with Sync Objects - Twilio <https://www.twilio.com/docs/sync/objects-overview>
2. Cloudflare Durable Objects vs Twilio - Ably Realtime <https://ably.com/compare/cloudflare-durable-objects-vs-twilio>
3. Introducing Sync – Real-time State Synchronization API | Twilio <https://www.twilio.com/en-us/blog/products/launches/sync-html>
4. [PDF] Designing a User-Friendly Inventory Management System Interface ... <https://www.aasmr.org/liss/Vol.11/No.10/Vol.11.No.10.03.pdf>
5. Designing for People with Low Digital Literacy - Thoughtworks <https://www.thoughtworks.com/insights/blog/designing-people-low-digital-literacy>
6. Designing for Low Digital Literacy Users - UX Bulletin <https://www.ux-bulletin.com/designing-low-digital-literacy-users/>
7. Self-Service Kiosk Printers & Hardware - Star Micronics https://starmicronics.com/self-service-kiosk-printer-solutions/?srsltid=AfmBOoovFGr0FPa-wUGWMAuzIs_KdEWpD9BlJfyo14hj1OMCHuDXHz9L
8. Thermal Printing Solutions for the Kiosk Industry - APS Printers <https://www.aps-printers.com/kiosk-industry-thermal-printer/>
9. Enhancing Self-Service Efficiency | Star Micronics Kiosk Printers <https://star-emea.com/kiosk-printers-star-micronics-self-service/>
10. Window: beforeinstallprompt event - Web APIs - MDN Web Docs https://developer.mozilla.org/en-US/docs/Web/API/Window/beforeinstallprompt_event
11. Trigger installation from your PWA - Progressive web apps | MDN https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/How_to/Trigger_install_prompt
12. Progressive Web Apps 2026: Complete Development Guide <https://www.digitalapplied.com/blog/progressive-web-apps-2026-complete-development-guide>
13. Add to Home screen - Progressive web apps (PWAs) | MDN https://mdn2.netlify.app/en-us/docs/web/progressive_web_apps/add_to_home_screen/
14. How to Implement Cloudflare Durable Objects - OneUptime <https://oneuptime.com/blog/post/2026-01-27-cloudflare-durable-objects/view>

15. Trying to use Websocket Hibernation Api - Stack Overflow <https://stackoverflow.com/questions/79336461/trying-to-use-websocket-hibernation-api>
16. What are Durable Objects? - Cloudflare Docs <https://developers.cloudflare.com/durable-objects/concepts/what-are-durable-objects/>
17. What even are Cloudflare Durable Objects? - Boris Tane <https://boristane.com/blog/what-are-cloudflare-durable-objects/>
18. Zero-latency SQLite storage in every Durable Object - Hacker News <https://news.ycombinator.com/item?id=41832547>
19. Chapter 25: Migration Playbooks | Architecting on Cloudflare <https://architectingoncloudflare.com/chapter-25>
20. Multi-Tenant Platform Development - Cloudflare <https://www.cloudflare.com/solutions/platforms/>
21. Chapter 23: Multi-Tenant and Platform Architectures <https://architectingoncloudflare.com/chapter-23/>
22. Workers for Platforms - Programmable Solutions - Cloudflare <https://www.cloudflare.com/products/workers-for-platforms/>
23. How are per-tenant/per-user/per-entity databases intended to be ... https://www.reddit.com/r/CloudFlare/comments/1itaza2/how_are_pertenantperuserperentity_databases/
24. Limits · Cloudflare D1 docs <https://developers.cloudflare.com/d1/platform/limits/>
25. FAQs · Cloudflare D1 docs <https://developers.cloudflare.com/d1/reference/faq/>
26. Top 50 Advanced Full Stack Developer Interview Questions ... <https://digitaldefynd.com/IQ/full-stack-developer-interview-questions/>
27. AI vs Human Cost Efficiency in Call Centers | PDF - Scribd <https://www.scribd.com/document/903682546/ChatGPT-AI-vs-Human-Cost-LLM-Orchestrator>
28. SQLite em produção? Eu ri da ideia. Até ver os números. Passei ... <https://www.instagram.com/p/DY7bMDwFDQU/>
29. Progressive Web Apps - web.dev <https://web.dev/learn/pwa/progressive-web-apps>
30. Window: localStorage property - Web APIs | MDN <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
31. Window: navigator property - Web APIs - MDN Web Docs <https://developer.mozilla.org/en-US/docs/Web/API/Window/navigator>
32. ASP.NET Core Web API printing to USB thermal printer on remote ... <https://stackoverflow.com/questions/79927490/asp-net-core-web-api-printing-to-usb-thermal-printer-on-remote-kiosk>

33. Expedy Print - Cloud Print Server for Developers <https://www.expedy.io/en?srsltid=AfmBOoq9i2O8dKWJp4JQ3gDQ3Zc08ioQz5hIGi62KpFxZKPSM5wo6Ufa>
34. Healthcare staff digital literacy levels and their attitudes towards ... https://www.researchgate.net/publication/332426684_Healthcare_staff_digital_literacy_levels_and_their_attitudes_towards_information_systems
35. Digital literacy level and associated factors among health ... - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC10126829/>
36. Cloudflare Workers Scaffold Example — Threat Model, ADRs, Test ... <https://stackblder.com/sample-scaffold>
37. PWA Browser Scorecards - Hacker News <https://news.ycombinator.com/item?id=45521704>
38. How to store/stash JavaScript event and reuse it later? <https://stackoverflow.com/questions/50461565/how-to-store-stash-javascript-event-and-reuse-it-later>
39. How to Build a Fully Local Thermal Printer Server (No Cloud ... <https://dev.to/alanwest/how-to-build-a-fully-local-thermal-printer-server-no-cloud-required-3a7k>
40. CloudPRNT Next for Developers: POS Receipt and Cloud Printing https://starmicronics.com/cloudprnt-web-cloud-online-pos-receipt-printing-sdk-developers?srsltid=AfmBOopWBXt8gBM4UzBNK_32s2GfPa8KZun0LwnMTylhpVoQypYfgIQ7
41. Role-Based Multi-Tenant Fron-tend Architecture in Angular <https://dev.to/elviskim18/role-based-multi-tenant-fron-tend-architecture-in-angular-4mg2>
42. Overview · Cloudflare Durable Objects docs <https://developers.cloudflare.com/durable-objects/>
43. Durable Objects (DO) — Unlimited single-threaded servers spread ... <https://www.lambrospetrou.com/articles/durable-objects-cloudflare/>
44. Chapter 11: Choosing the Right Storage | Architecting on Cloudflare <https://architectingoncloudflare.com/chapter-11>
45. CloudPRNT Next for Developers: POS Receipt and Cloud Printing <https://starmicronics.com/cloudprnt-web-cloud-online-pos-receipt-printing-sdk-developers?srsltid=AfmBOooNR2pJ8j4JrBSMOtr7q5Bc1tSyz7ZkCwsUssODEiD4y3fgvL9x>
46. Data location - Durable Objects - Cloudflare Docs <https://developers.cloudflare.com/durable-objects/reference/data-location/>
47. Building Real-Time Chat in Astro with Cloudflare Durable Objects ... <https://www.launchfa.st/blog/real-time-chat-astro-cloudflare-durable-objects>
48. Build a WebSocket server - Durable Objects - Cloudflare Docs <https://developers.cloudflare.com/durable-objects/examples/websocket-server/>

49. Building Waiting Room on Workers and Durable Objects <https://blog.cloudflare.com/building-waiting-room-on-workers-and-durable-objects/>
50. Building Real-Time Dashboards with Cloudflare Durable Objects <https://andrewusher.dev/blog/building-realtime-dashboards-with-durable-objects>
51. Serverless Progressive Web Apps using React with Cloudflare ... <https://blog.cloudflare.com/serverless-pwa-react-cloudflare-workers/>
52. How to Create SMS Handling System | Mobile Security Research <https://www.corellium.com/blog/create-sms-handling-system-mobile-security>
53. Verifying JWTs in Cloudflare Workers - Kinde docs <https://docs.kinde.com/integrate/third-party-tools/verifying-jwts-cloudflare-workers/>
54. JSON Web Tokens validation - API Shield - Cloudflare Docs <https://developers.cloudflare.com/api-shield/security/jwt-validation/>
55. How to design equitable digital health tools: A narrative review of ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC11340894/>
56. Durable Objects, now in Open Beta - The Cloudflare Blog <https://blog.cloudflare.com/durable-objects-open-beta/>
57. BeforeInstallPromptEvent - Web APIs | MDN <https://developer.mozilla.org/en-US/docs/Web/API/BeforeInstallPromptEvent>
58. Use WebSockets · Cloudflare Durable Objects docs <https://developers.cloudflare.com/durable-objects/best-practices/websockets/>
59. Debugging WebSocket Hibernation with Cloudflare Durable Objects ... <https://thomasgauvin.com/writing/how-cloudflare-durable-objects-websocket-hibernation-works/>
60. Expedy Print - Cloud Print Server for Developers <https://www.expedy.io/en?srsltid=AfmBOoo--OSlFJ9moGy9UM1QHTjBIkGD1lpz3gSxqEehJsYjN3ovkgRG>
61. CloudPRNT Next for Developers: POS Receipt and Cloud Printing https://starmicronics.com/cloudprnt-web-cloud-online-pos-receipt-printing-sdk-developers/?srsltid=AfmBOorS53QuhlRx_zvSV76phWDFM3zhLflih8mlEB13qfa0JB9yf382
62. Printercow <https://www.printercow.com/>
63. Installation - What PWA Can Do Today <https://whatpwacando.today/installation/>
64. Supercharge Your Website Using PWA: Deferred Installation <https://dev.to/ruppysupply/supercharge-your-website-using-pwa-deferred-installation-nhb>
65. PWA install criterias for beforeinstallprompt event? - Reddit https://www.reddit.com/r/PWA/comments/1qw9aja/pwa_install_criterias_for_beforeinstallprompt/
66. Install pwa from a button - Ask us anything - WeWeb Community <https://community.weweb.io/t/install-pwa-from-a-button/6114>

67. PWA - Prevent logging in a second time on iOS using localStorage https://www.reddit.com/r/PWA/comments/1kgtgb3/pwa_prevent_logging_in_a_second_time_on_ios_using/
68. PWA iOS Limitations and Safari Support [2026] - MagicBell <https://www.magicbell.com/blog/pwa-ios-limitations-safari-support-complete-guide>
69. iOS Safari/WebKit current state of PWA persistence as of version 15.2 <https://stackoverflow.com/questions/71783032/ios-safari-webkit-current-state-of-pwa-persistence-as-of-version-15-2>
70. Getting 'Save to Home Screen' to Kinda Work on iOS - naildrivin5.com <https://naildrivin5.com/blog/2023/08/24/braindump-of-pwa-on-ios.html>
71. PWA on iOS: Install Guide & Limits for Advertisers 2026 - DeepClick <https://deepclick.com/resources/blog/progressive-web-apps-on-ios/>
72. Do Progressive Web Apps Work on iOS? The Complete Guide for ... <https://www.mobiloud.com/blog/progressive-web-apps-ios>
73. Does iOS support the "beforeinstallprompt" event? - Stack Overflow <https://stackoverflow.com/questions/55302527/does-ios-support-the-beforeinstallprompt-event>
74. Deploy a real-time chat application · Cloudflare Workers docs <https://developers.cloudflare.com/workers/tutorials/deploy-a-realtime-chat-app/>
75. Durable Objects llms-full.txt - Cloudflare Docs <https://developers.cloudflare.com/durable-objects/llms-full.txt>
76. Release notes - Durable Objects - Cloudflare Docs <https://developers.cloudflare.com/durable-objects/release-notes/>
77. Durable Objects: Easy, Fast, Correct — Choose three <https://blog.cloudflare.com/durable-objects-easy-fast-correct-choose-three/>